
USRT ILP v2

Utility-aware Energy-Constrained Real-Time Scheduling

SPS Mapping + ILP Optimisation

Core Idea: Decompose the USRT joint-optimisation into two sequential phases.

Phase 1 SPS Mapping $\text{job} \rightarrow \text{processor}$ $\xrightarrow{\text{mapping } P_{i,j}^* \text{ fixed}}$ Phase 2 ILP $\text{freq} + \text{segments}$

Phase 1 uses **SPS** ($O(n \log n)$) to assign jobs to processors via utilisation-balanced load.

Phase 2 solves a smaller ILP for frequency and segment count, with the mapping fixed.

ILP v1 — Joint Optimisation

Variable $V(i, j, k, \mathbf{x}, z)$ — huge search

Optimal — very slow for large n

ILP v2 — Decomposed

Variable $Y(i, j, k, z)$ — no x dim.

Sub-optimal — fast & scalable

Contents

1	Problem Overview	2
1.1	Why ILP v2?	2
2	System Model	2
2.1	Processor, Task & Job Model	3
2.2	Cumulative Execution Time	3
2.3	Energy Model	3
2.4	Utility Model	4
3	ILP v2 Formulation	5
3.1	Decision Variable	5
3.2	Constraints	5
3.3	Objective	6
3.4	Complete Formulation Summary	6
4	Phase 1 — Quantum SPS Mapping	7
5	Correctness & Feasibility	12
5.1	Mandatory Feasibility Guarantee	12
5.2	Constraint Enforcement	12
5.3	Why ILP v2 May Give Less Utility Than ILP v1	12
6	Algorithm Flow Summary	13
7	Function Reference	13
8	Complexity	14
	References	14

Problem Overview

The **USRT** (Utility-aware Scheduling of Real-Time tasks) problem maps N_{tsk} periodic harmonic tasks across N_{prc} homogeneous processors under a global energy budget B , maximising total utility from optional segment execution. Execution is **preemptive EDF** at the **job level**.

Why ILP v2?

ILP v1 solves the problem *jointly* with decision variable $V(i, j, k, x, z)$ — choosing processor x , frequency f_z , and segment count k simultaneously. This gives the global optimum but becomes computationally intractable as the number of jobs grows.

Aspect	ILP v1	ILP v2 (This Document)
Decision variable	$V(i, j, k, x, z) \in \{0, 1\}$ — joint	$Y(i, j, k, z) \in \{0, 1\}$ — no x
Mapping	Decided by ILP (optimal)	Fixed by SPS (heuristic)
Freq + seg-ments	Decided by ILP	Decided by ILP
Solution quality	Global optimum	Sub-optimal (SPS may not be optimal)
Scalability	Poor for large instances	Good — practical
Variable count	$N_{\text{jobs}} \times (N_{\text{seg}} + 1) \times N_{\text{prc}} \times N_{\text{frq}}$	$N_{\text{jobs}} \times (N_{\text{seg}} + 1) \times N_{\text{frq}}$

Key insight: Once the mapping $P_{i,j}^*$ is fixed by SPS, the per-processor scheduling problems become *decoupled* — the ILP only needs to decide frequency and segment count for each job on its assigned processor. The search space shrinks by a factor of N_{prc} .

System Model

Processor, Task & Job Model

Symbol	Meaning
p_i	Period of task T_i (harmonic: $p_i \mid p_j$ or $p_j \mid p_i$)
$r_{i,j} = (j-1) \cdot p_i$	Release time of job $T_{i,j}$
$d_{i,j} = j \cdot p_i$	Deadline of job $T_{i,j}$
$h = \text{lcm}\{p_i\}$	Hyper-period; $N_i^{\text{job}} = h/p_i$ jobs per task
$e_{i,0}$	Mandatory segment execution time at $f_{\max}$ (stored as <code>e_m</code>)
$e_{i,k} \ (k \geq 1)$	Cumulative exec time including k optional segments at $f_{\max}$ (stored in <code>e_o_k</code>)
N_i^{seg}	Number of optional segments for task T_i
$G_{i,k}$	The k^{th} segment; executing $G_{i,k}$ implies $G_{i,0}, \dots, G_{i,k-1}$ are done
u_i	Utility per unit optional execution time for task T_i
$\mathcal{F} = \{f_1, \dots, f_{\max}\}$	Discrete normalised frequency set; $f_{\max} = 1.0$

Cumulative Execution Time

In code, `cum[i][k]` stores cumulative execution time up to segment k at $f_{\max}$:

Definition — Cumulative Exec Time (`build_cum`)

$$\text{cum}[i][k] = \sum_{q=0}^k e_{i,q} \quad k \in \{0, 1, \dots, N_i^{\text{seg}}\} \quad (1)$$

$\text{cum}[i][0] = e_{i,0}$ (mandatory only). $\text{cum}[i][k] - \text{cum}[i][0] =$ total optional execution up to segment k .

Energy Model

Equation 1 — Effective Execution Time

$$e_{i,j,k,z}^{\text{eff}} = \frac{\text{cum}[i][k]}{f_z} \quad (2)$$

Equation 2 — Energy Consumption

$$E_{i,k,z} = \alpha \cdot \frac{\text{cum}[i][k]}{f_z} + \beta \cdot f_z^2 \cdot \text{cum}[i][k] \quad (3)$$

Default: $\alpha = 1.0$, $\beta = 0.5$. Note $E_{i,k,z}$ is *independent of job index j* — energy depends only on task i , segment level k , and frequency f_z .

Energy minimum at $f_{\max}$: Setting $\partial E / \partial f_z = 0$ gives $f_{\text{opt}} = (\alpha/2\beta)^{1/3} = 1.0 = f_{\max}$ for $\alpha = 1, \beta = 0.5$. Decreasing frequency *always increases energy* for these defaults. The ILP will therefore never set $z < N_{\text{frq}} - 1$ when energy is the binding constraint.

Utility Model

Equation 3 — Job Utility

$$U_{i,j}^{\text{job}} = u_i \times (\text{cum}[i][k] - \text{cum}[i][0]) \quad (k \geq 1) \quad (4)$$

Total utility objective: $\Phi = \sum_i \sum_j U_{i,j}^{\text{job}}$

ILP v2 Formulation

The ILP v2 formulation is taken directly from Section IV of the USRT paper.

Decision Variable

The mapping $P_{i,j}^*$ is fixed by SPS (Phase 1). Given this, define:

ILP v2 Decision Variable

$Y(i, j, k, z) \in \{0, 1\}$ = 1 iff job $T_{i,j}$ executes with frequency f_z up to segment k on processor $P_{i,j}^*$ (fixed by SPS)

Indices: $i \in [1, N_{\text{tsk}}]$, $j \in [1, N_i^{\text{job}}]$, $k \in [0, N_i^{\text{seg}}]$, $z \in [1, N_{\text{freq}}]$

Key difference from ILP v1: $Y(i, j, k, z)$ has *no processor index x* . The processor is already determined: $T_{i,j}$ runs on $P_{i,j}^*$ as decided by SPS.

Constraints

C1 — Assignment Constraint (Eq. 11)

Each job executes with exactly one frequency and up to zero or more optional segments:

C1 — Exactly One (k, z) per Job

$$\sum_{k=0}^{N_i^{\text{seg}}} \sum_{z=1}^{N_{\text{freq}}} Y(i, j, k, z) = 1 \quad \forall i, j \quad (5)$$

C2 — Timing Constraint / DBF (Eq. 12–13)

Let $\mathcal{A}_x = \{r_{i,j} : P_{i,j}^* = x\}$ and $\mathcal{D}_x = \{d_{i,j} : P_{i,j}^* = x\}$ be the arrival and deadline sets for jobs on processor P_x .

For processor P_x and window $[t_1, t_2]$ with $t_1 \in \mathcal{A}_x$, $t_2 \in \mathcal{D}_x$, define:

$$S(t_1, t_2, x) = \{T_{i,j} : P_{i,j}^* = x, r_{i,j} \geq t_1, d_{i,j} \leq t_2\}$$

C2 — Demand Bound Function (DBF)

$$\text{DBF}(t_1, t_2, x) = \sum_{T_{i,j} \in S(t_1, t_2, x)} \sum_{k,z} Y(i, j, k, z) \cdot e_{i,j,k,z}^{\text{eff}} \leq t_2 - t_1 \quad \forall x, t_1 \in \mathcal{A}_x, t_2 \in \mathcal{D}_x \quad (6)$$

Important difference from ILP v1: $\mathcal{A}_x$ and $\mathcal{D}_x$ include *only jobs mapped to P_x* , not all jobs. This dramatically reduces the number of DBF constraints.

C3 — Energy Budget

C3 — Global Energy Constraint

$$E^{\text{total}} = \sum_{i,j,k,z} Y(i,j,k,z) \cdot E_{i,k,z} \leq B \quad (7)$$

Energy $E_{i,k,z}$ is from Eq. (3). Note: The ILP uses auxiliary variable $Y(i,j,k,z) = \sum_x V(i,j,k,x,z)$ from v1 but here Y is directly the primary variable since processor is fixed.

Objective

Objective — Maximise Total Utility (Eq. 8–10)

$$\Phi = \sum_i \sum_j U_{i,j}^{\text{job}}$$

$$\text{where } U_{i,j}^{\text{job}} = u_i \times \sum_{\substack{k=1 \\ z}}^{N_i^{\text{seg}}} Y(i,j,k,z) \times (\text{cum}[i][k] - \text{cum}[i][0]) \quad (8)$$

Only $k \geq 1$ contributes utility (optional segments only). At $k = 0$ (mandatory only): $\text{cum}[i][0] - \text{cum}[i][0] = 0$.

Complete Formulation Summary

Label	Expression	Role
C1	$\sum_{k,z} Y(i,j,k,z) = 1 \quad \forall i,j$	Each job gets exactly one (k,z) .
C2	$\text{DBF}(t_1, t_2, x) \leq t_2 - t_1 \quad \forall x, t_1 \in \mathcal{A}_x, t_2 \in \mathcal{D}_x$	EDF timing feasibility per processor.
C3	$\sum_{i,j,k,z} Y(i,j,k,z) \cdot E_{i,k,z} \leq B$	Global energy budget.
Obj	$\max_{\text{cum}[i][0]} \sum_{i,j} u_i \cdot \sum_{k \geq 1, z} Y(i,j,k,z) \cdot (\text{cum}[i][k] - \text{cum}[i][0])$	Maximise total utility.

Phase 1 — Quantum SPS Mapping

Phase 1 — Quantum SPS Mapping

Functions: `quantum_sps_mapping()` `run_dps()` `run_sps()` `check_dbf()`
`repair_mapping()` `generate_jobs()`

Goal

Produce a feasible job-to-processor mapping $P_{i,j}^*$ using the SPS algorithm, ensuring that the per-processor utilisation satisfies the EDF schedulability condition. The mapping is produced quantum by quantum at the GCD timescale.

Load Metric — Why Utilisation?

Why utilisation $u_i = e_m/p_i$ and NOT $C_{\max} \leq \text{GCD}$?

For *preemptive* scheduling, a job with $e_m = 20.63$ and $p = 90$ does not need to finish within a single 15-unit quantum. It spreads its execution across its full 90-unit period. The correct feasibility criterion for preemptive EDF (Liu & Layland, 1973) for harmonic periods is:

$$\sum_{T_{i,j} \rightarrow P_x} \frac{e_{m_i}}{p_i} \leq 1.0 \quad \forall x$$

A $C_{\max} \leq \text{GCD}$ check would incorrectly reject this job (since $20.63 > 15$), causing it to be force-mapped via round-robin, leading to DBF violations and an infeasible ILP.

Quantum Classification

The hyper-period $h = \text{lcm}\{p_i\}$ is divided into quanta of length $\delta = \text{gcd}\{p_i\}$. At quantum $[q, q + \delta]$:

Class	Condition	Treatment
Mandatory	$d_{i,j} = q + \delta$	Must be mapped this quantum or deadline is missed.
Optional	$d_{i,j} > q + \delta$	Can be deferred; sorted by deadline ascending.
Leftover	Carried from prior quanta	Mixed with newly arrived jobs.

Per-Quantum Algorithm

Algorithm 1 Per-Quantum SPS Mapping (one quantum $[q, q + \delta]$)

Input: pending jobs, current `proc_util` []

Output: mapping for this quantum; updated `proc_util` []

```

1: mandatory  $\leftarrow \{(i, j, r, d) : d = q + \delta\}$ 
2: optional  $\leftarrow \{(i, j, r, d) : d > q + \delta\}$ , sorted by  $d$  ascending
3: active  $\leftarrow$  mandatory + optional
4: while true do
5:    $U_{\text{active}} \leftarrow \sum_{(i,j) \in \text{active}} e_{m_i}/p_i$ 
6:    $C_{\text{rem}} \leftarrow \sum_x (1.0 - \text{proc\_util}[x])$ 
7:   if  $U_{\text{active}} > C_{\text{rem}}$  then ▷ Pre-filter: even perfect balance won't fit
8:     Defer latest-deadline optional job to leftover pool
9:   else
10:    Run DPS + SPS on active set
11:    Compute tentative new_util []
12:    if  $\max(\text{new\_util}) \leq 1.0$  then
13:      Commit mapping; proc_util  $\leftarrow$  new_util; break
14:    else
15:      if no optional jobs remain then
16:        Commit anyway (overloaded; global DBF repair will handle)
17:      else
18:        Defer latest-deadline optional job to leftover pool
```

DPS Procedure — Determining Partial Solutions

Algorithm 2 DPS — Determining Partial Solutions

Input: Jobs with loads $\{(i, j), e_{m_i}/p_i\}$ sorted descending; m processors

Output: Almost-balanced p -family $\mathcal{S}$

```

1:  $\mathcal{S} \leftarrow \{\text{PS.singleton}(\text{first job})\}$ 
2: for all remaining jobs  $(i, j)$  with load  $v$  do
3:    $r \leftarrow \arg \max_{r'} \Delta_{r'}$  ▷ partial solution with largest gap
4:   if  $v \leq \Delta_r$  then
5:     Insert  $(i, j)$  into least-loaded machine of  $S_r$ ; re-sort loads
6:   else
7:      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{PS.singleton}(i, j)\}$ 
return  $\mathcal{S}$ 
```

SPS Combination

Algorithm 3 SPS — Ordered Combination

Input: Almost-balanced p -family $\mathcal{S}$

Output: Single final mapping

```

1: while  $|\mathcal{S}| > 1$  do
2:   Select  $S_l, S_k$  with two largest gaps  $\Delta_l, \Delta_k$ 
3:   Machine  $j$  of new PS  $\leftarrow S_l[j] \cup S_k[m - 1 - j]$ , then re-sort ▷ anti-sorted pairing
   minimises gap
4:    $\mathcal{S} \leftarrow (\mathcal{S} \setminus \{S_l, S_k\}) \cup \{S_{\text{new}}\}$ 
return  $\mathcal{S}[0]$ 
```

Gap non-increasing property (Prop. 2.6, Chiaselotti et al.): The gap of the combined partial solution satisfies $\Delta_{S_l \triangleright \triangleleft S_k} \leq \max(\Delta_l, \Delta_k)$. This guarantees the final mapping is well-balanced.

Post-Mapping: DBF Check & Repair

After all quanta are processed:

1. **Global DBF check** (`check_dbf()`): verify every window $[t_1, t_2]$ on every processor satisfies DBF using e_m only (mandatory at $f_{\max}$). If this passes, the ILP is guaranteed to have at least one feasible solution ($k = 0, z = f_{\max}$ for all jobs).
2. **Greedy repair** (`repair_mapping()`): if violations exist, find the worst-violated (processor, window), move the job with largest e_m in that window to the least-loaded other processor. Repeat up to 100 iterations.

Phase 2 — ILP Optimisation — Frequency & Segment Assignment

Functions: `solve_ilp_v2()` `build_cum()` `generate_jobs()`

Setup

With mapping $P_{i,j}^*$ fixed, group jobs by processor:

$$J_x = \{(i, j) : P_{i,j}^* = x\} \quad \forall x \in \{0, \dots, N_{\text{prc}} - 1\}$$

Precompute:

- $\mathcal{A}_x = \{r_{i,j} : (i, j) \in J_x\}$ — arrival times on processor x
- $\mathcal{D}_x = \{d_{i,j} : (i, j) \in J_x\}$ — deadline times on processor x
- `cum[i][k]` for all i, k via `build_cum()`

Variable Count

Formulation	Variable	Count
ILP v1	$V(i, j, k, x, z)$	$N_{\text{jobs}} \times (N_{\text{seg}} + 1) \times N_{\text{prc}} \times N_{\text{frq}}$
ILP v2	$Y(i, j, k, z)$	$N_{\text{jobs}} \times (N_{\text{seg}} + 1) \times N_{\text{frq}}$
Reduction factor		N_{prc} (one dimension eliminated)

Constraint C1 in Code

```
for i in range(N_tsk):
    for j in range(N_job[i]):
        mdl.addConstr(
            gp.quicksum(Y[i, j, k, z]
                        for k in range(N_seg[i]+1)
                        for z in range(N_frq)) == 1,
            name=f"C1_{i}_{j}"
        )
```

Constraint C2 in Code

Note that $\mathcal{A}_x$ and $\mathcal{D}_x$ are built from jobs on P_x only:

```
for x in range(N_prc):
    jobs_x = proc_jobs[x]
    Ax = sorted({job_r[ij] for ij in jobs_x})
    Dx = sorted({job_d[ij] for ij in jobs_x})
    for t1 in Ax:
        for t2 in Dx:
            if t1 >= t2: continue
            window = [(i, j) for (i, j) in jobs_x
                      if job_r[(i, j)] >= t1 and job_d[(i, j)] <= t2]
            if not window: continue
            mdl.addConstr(
                gp.quicksum(Y[i, j, k, z] * e_eff(i, k, z)
                            for (i, j) in window
                            for k in range(N_seg[i]+1)
                            for z in range(N_frq)) <= t2 - t1
            )
```

DBF constraint count reduction: In ILP v2, each processor's DBF constraints only involve jobs mapped to that processor. If jobs are spread evenly, $|\mathcal{A}_x| \approx N_{\text{arrivals}}/N_{\text{prc}}$ and $|\mathcal{D}_x| \approx N_{\text{deadlines}}/N_{\text{prc}}$, so the number of DBF constraints shrinks by up to N_{prc}^2 .

Constraint C3 in Code

```
mdl.addConstr(
    gp.quicksum(Y[i,j,k,z] * E(i,k,z)
                for i in range(N_tsk)
                for j in range(N_job[i])
                for k in range(N_seg[i]+1)
                for z in range(N_frq)) <= B_BUDGET,
    name="C3_energy"
)
```

Objective in Code

```
mdl.setObjective(
    gp.quicksum(
        tasks[i]['u_i'] * (cum[i][k] - cum[i][0]) * Y[i,j,k,z]
        for i in range(N_tsk)
        for j in range(N_job[i])
        for k in range(1, N_seg[i]+1)    # k=0 gives zero utility
        for z in range(N_frq)
    ),
    GRB.MAXIMIZE
)
```

Note: $\text{cum}[i][0] - \text{cum}[i][0] = 0$ for $k = 0$, so the loop starts at $k = 1$.

Correctness & Feasibility

Mandatory Feasibility Guarantee

If the global DBF check passes with e_m at $f_{\max}$ (after Phase 1), then setting $Y(i, j, k=0, z=N_{\text{frq}}-1) = 1$ for all (i, j) is always a feasible ILP solution. This means:

If Phase 1 produces a DBF-feasible mapping (mandatory only, $f_{\max}$), then Phase 2 ILP is always feasible. The ILP then optimises beyond this baseline to add optional segments.

Constraint Enforcement

Constraint	How Enforced	Function(s)
No deadline miss	Phase 1: utilisation ≤ 1.0 ensures EDF feasibility. Phase 2: C2 (DBF) is a hard ILP constraint.	<code>check_dbf()</code> <code>repair_mapping()</code>
Energy budget	C3 is a hard ILP constraint. Global energy never exceeds B .	<code>solve_ilp_v2()</code>
No migration	$P_{i,j}^*$ fixed after Phase 1; never changed in Phase 2.	<code>quantum_sps_mapping()</code>
Single frequency	C1 ensures exactly one (k, z) per job; z cannot change during execution.	<code>solve_ilp_v2()</code>
Mandatory exec.	$k = 0$ always feasible (C1 requires some selection). Optional segments are additive.	<code>C1 in solve_ilp_v2()</code>
EDF ordering	DBF feasibility $\Rightarrow$ EDF schedulability for harmonic periods.	<code>C2 in solve_ilp_v2()</code>

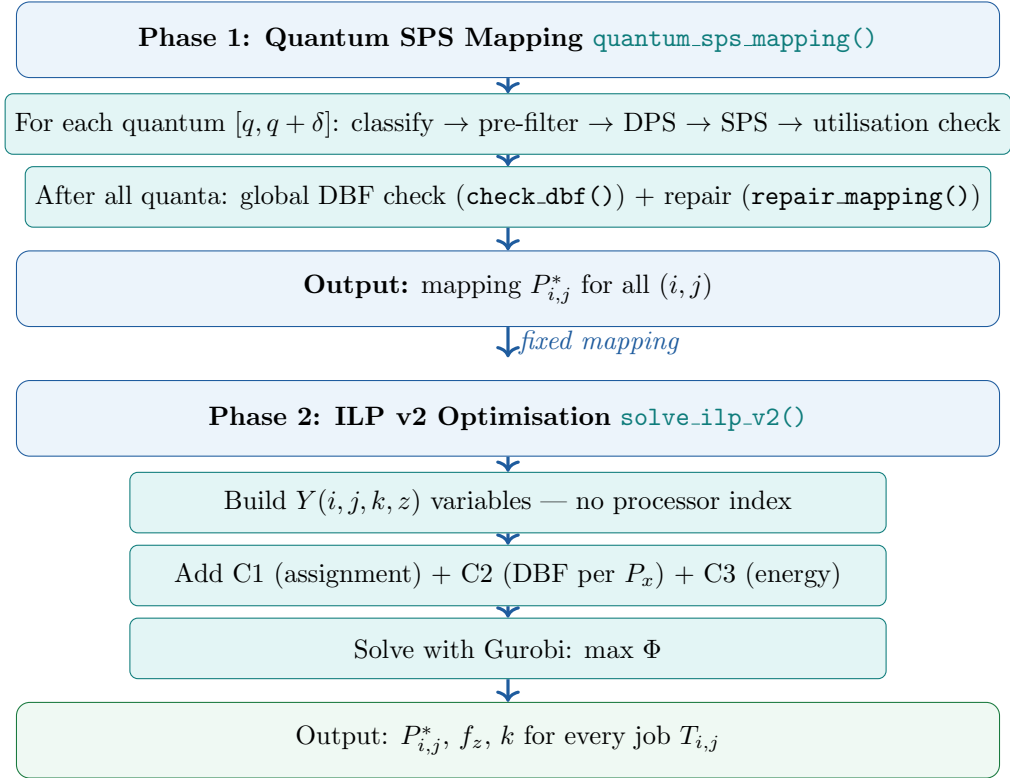
Why ILP v2 May Give Less Utility Than ILP v1

The utility gap arises because SPS fixes the mapping before seeing the ILP objective:

1. **Mapping ignores utility:** SPS balances e_m/p_i loads without knowing u_i or optional segment sizes. A high-utility job may land on a crowded processor with no DBF slack for its optional segments.
2. **Mapping ignores frequency:** In ILP v1, choosing a lower frequency for one job can shift its processor assignment to create slack elsewhere. In v2, the mapping is fixed before frequencies are chosen.
3. **DBF coupling:** In ILP v1, when job A chooses $k = 2$ and needs more time, the solver can simultaneously move job B to a less-loaded processor. In v2, job B's processor is already fixed.

The utility gap is the **price of scalability**: ILP v2 can solve much larger instances than ILP v1.

Algorithm Flow Summary



Function Reference

Function	Phase	Description
<code>quantum_sps_mapping()</code>	1	Top-level Phase 1 driver. Per-quantum loop, left-over handling, DBF check + repair.
<code>generate_jobs()</code>	1	Returns list of $(i, j, r_{i,j}, d_{i,j})$ for all jobs in the hyper-period.
<code>run_dps()</code>	1	Determining Partial Solutions: greedy insertion into almost-balanced partial solutions.
<code>run_sps()</code>	1	SPS combination: iteratively merge two PS with largest gaps until one remains.
<code>check_dbf()</code>	1	Global DBF feasibility check at $f_{\max}$, mandatory only. Returns (ok, violations).
<code>repair_mapping()</code>	1	Greedy repair: move heaviest offending job to least-loaded processor.
<code>solve_ilp_v2()</code>	2	Builds and solves the ILP with variables $Y(i, j, k, z)$; constraints C1/C2/C3.
<code>build_cum()</code>	util	$\text{cum}[i][k] = \sum_{q=0}^k e_{i,q}$ — cumulative exec times at $f_{\max}$.

Function	Phase	Description
<code>gcd_list()</code>	util	$\gcd\{p_i\}$ — quantum length.
<code>lcm_list()</code>	util	$\text{lcm}\{p_i\}$ — hyper-period h .
<code>load_testcase()</code>	util	Dynamically imports <code>testcase.py</code> and returns (<code>processors</code> , <code>tasks</code> , <code>B_BUDGET</code>).
<code>print_mapping_summary()</code>	out	Prints per-processor job list with utilisation and e.m.
<code>_print_solution()</code>	out	Prints ILP solution: per-job $(k, z, e^{\text{eff}}, E, U)$ and totals.

Complexity

Component	Complexity	Notes
DPS per quantum	$\mathcal{O}(n \log n)$	Sort + heap operations.
SPS combination	$\mathcal{O}(p^2)$	p = number of partial solutions; $p \leq n$.
Phase 1 total	$\mathcal{O}((h/\delta) \cdot n \log n)$	One DPS+SPS per quantum.
C1 constraints	$\mathcal{O}(N_{\text{jobs}})$	One per job.
C2 constraints	$\mathcal{O}(N_{\text{prc}} \cdot (N_{\text{jobs}}/N_{\text{prc}})^2)$	Per-processor window pairs; N_{prc} reduction vs v1.
ILP variables	$\mathcal{O}(N_{\text{jobs}} \cdot N_{\text{seg}} \cdot N_{\text{frq}})$	N_{prc} fewer than ILP v1.
ILP solve	Gurobi branch-and-bound	Exponential worst-case; fast for typical instances.

References

References

- [1] G. Chiaselotti, M.I. Gualtieri, P. Pietramala, “Minimizing the Makespan in Nonpreemptive Parallel Machine Scheduling Problem,” *Journal of Mathematical Modelling and Algorithms*, vol. 9, pp. 39–51, 2010.
- [2] USRT Problem Statement — Utility-aware Energy-Constrained Real-Time Scheduling, *Segmented Workload Scheduling Shared v2*, Section IV.
- [3] C.L. Liu, J.W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

- [4] N. Kumar, J. Mayank, A. Mondal, “Reliability Aware Energy Optimized Scheduling of Non-preemptive Periodic Real-Time Tasks on Heterogeneous Multiprocessor System,” *IEEE TPDS*, vol. 31, no. 4, pp. 871–885, 2019.